

Sauvegarde des données de jeu dans Unity

La sauvegarde des données de jeu est une fonctionnalité essentielle pour de nombreux jeux vidéo, permettant aux joueurs de conserver leur progression et leurs paramètres. Dans ce chapitre, nous allons explorer différentes méthodes pour sauvegarder et charger des données dans Unity. Nous aborderons les `PlayerPrefs`, la sérialisation JSON, et l'utilisation de fichiers binaires.

L'utilisation d'une base de données externe serait possible mais ne fait pas partie de ce cours.

1.1 PlayerPrefs

Unity fournit une classe intégrée appelée `PlayerPrefs` qui permet de sauvegarder des données simples telles que des entiers, des flottants et des chaînes de caractères. Il est possible d'enregistrer des données en utilisant une clé unique pour chaque valeur. Si vous avez besoin de sauvegarder des données plus complexes, vous devrez envisager d'autres méthodes comme la sérialisation JSON ou les fichiers binaires.

À noter que `PlayerPrefs` est principalement destiné à stocker des données de petite taille, comme les paramètres du jeu ou les scores élevés. Pour des données plus volumineuses, il est recommandé d'utiliser des méthodes de sauvegarde plus robustes et que ces données peuvent être facilement modifiées par l'utilisateur en fouillant dans les fichiers de l'application.

```
PlayerPrefs.SetInt("HighScore", 100);
PlayerPrefs.SetFloat("Volume", 0.8f);
PlayerPrefs.SetString("PlayerName", "Maxime");
```

Pour récupérer les données sauvegardées, vous pouvez utiliser les méthodes correspondantes :

```
int highScore = PlayerPrefs.GetInt("HighScore", 0);
float volume = PlayerPrefs.GetFloat("Volume", 1.0f);
string playerName = PlayerPrefs.GetString("PlayerName", "Unknown");
```

1.2 Sérialisation JSON

Pour sauvegarder des données plus complexes, vous pouvez utiliser la sérialisation JSON. Unity fournit la classe `JsonUtility` qui facilite la conversion d'objets en chaînes JSON et vice versa. Les données sérialisées peuvent ensuite être sauvegardées dans des fichiers sur le disque. Comme `playerPrefs`, cette méthode est simple à utiliser et les fichiers JSON sont lisibles, ce qui facilite le débogage mais aussi la modification par l'utilisateur. Vous pourriez avoir plusieurs fichiers de sauvegarde pour différencier les profils de joueurs.

Voici un exemple de comment sauvegarder et charger des données de jeu en utilisant JSON :

```
[System.Serializable]
public class GameData
{
```

```
    public int level;
    public float health;
    public string playerName;
}

public void SaveGame(GameData data, int saveSlot)
{
    string json = JsonUtility.ToJson(data);
    System.IO.File.WriteAllText($"{Application.persistentDataPath}/monJeu_save{saveSlot}.json", json);
}

public GameData LoadGame(int saveSlot)
{
    string path = $"{Application.persistentDataPath}/monJeu_save{saveSlot}.json";
    if (System.IO.File.Exists(path))
    {
        string json = System.IO.File.ReadAllText(path);
        return JsonUtility.FromJson<GameData>(json);
    }
    return null;
}
```

Dans un script dédié, vous pouvez appeler les fonctions SaveGame et LoadGame pour gérer la sauvegarde et le chargement des données de jeu.

```
Start(){
    GameData data = new GameData();
    int currentLevel = 1;
    float playerHealth = 100f;
    string playerName = "Hero";

    GameData loadedData = LoadGame();
    if (loadedData != null)
    {
        data = loadedData;

        currentLevel = data.level;
        playerHealth = data.health;
        playerName = data.playerName;
    }
}

EnregisterPartie(){

    data.level = currentLevel;
    data.health = playerHealth;
    data.playerName = playerName;

    SaveGame(data);
}
```

```
}
```

1.2.1 Application.persistentDataPath

Application.persistentDataPath est un chemin d'accès sûr pour stocker les fichiers de sauvegarde, accessible sur toutes les plateformes supportées par Unity.

1.3 Fichiers binaires

Pour des performances optimales et une sécurité accrue, vous pouvez utiliser la sérialisation binaire pour sauvegarder les données de jeu. Cette méthode est plus complexe à mettre en œuvre mais permet de stocker des données de manière plus compacte et moins lisible par l'utilisateur.

Voici un exemple de comment sauvegarder et charger des données en utilisant la sérialisation binaire :

```
using System.IO;
using System.Runtime.Serialization.Formatters.Binary;
[System.Serializable]
public class GameData
{
    public int level;
    public float health;
    public string playerName;
}

public void SaveGame(GameData data)
{
    BinaryFormatter formatter = new BinaryFormatter();
    string path = $"{Application.persistentDataPath}/monJeu_save.dat";
    FileStream stream = new FileStream(path, FileMode.Create);
    formatter.Serialize(stream, data);
    stream.Close();
}

public GameData LoadGame()
{
    string path = $"{Application.persistentDataPath}/monJeu_save.dat";
    if (File.Exists(path))
    {
        BinaryFormatter formatter = new BinaryFormatter();
        FileStream stream = new FileStream(path, FileMode.Open);
        GameData data = formatter.Deserialize(stream) as GameData;
        stream.Close();
        return data;
    }
    return null;
}
```

[Documentation Unity sur JsonUtility](#) [Documentation Unity sur PlayerPrefs](#)